

Technics SX-WSA1R

System Exclusive Reference

Acoustic Modeling Synthesizer Module

Operating system version 2.0

Unofficial

This document is not published, endorsed or checked by Technics or Panasonic. It was written by inspecting a disassembly of the SX-WSA1R's program ROMs, operating system version 2, and checking the result against a bulk dump captured from a real instrument.

Contents

1	Overview	3
1.1	Applicability	3
1.2	Ports	3
1.3	Relationship to the published documentation	3
1.4	Verification	4
1.5	Conventions	5
2	At a glance	6
2.1	Frame	6
2.2	Messages	6
2.3	Rules	6
2.4	Bulk dump	7
2.5	Errors	7
3	Message framing	8
3.1	Identifiers	8
3.2	Termination	8
3.3	Length limit	8
4	The Technics message set	9
4.1	Fields	9
4.2	Commands	9
4.3	Which fields each message carries	10
4.4	Tempo	11
4.5	Parameter commands	11
5	Bulk dump	12
5.1	Categories and what each transmits	12
5.2	Data message — BTR	13
5.3	Payload encoding	14
5.4	Checksum	14
5.5	Requesting a dump — DRQ	14
5.6	Opening a transfer	14
5.7	Acknowledgement and retry	15
5.8	Continuation — CDD	15
5.9	Closing	15
5.10	Receiving	15

6	What a block contains	16
6.1	Signatures	16
6.2	Names	16
7	Parameter messages	18
7.1	The address	18
7.2	Parts	18
7.3	What the instrument actually accepts	19
7.4	Parameters and the bulk dump are the same bytes	19
7.5	What a parameter accepts	20
7.6	Names	23
7.7	Whole-category request	24
7.8	The sound areas	24
8	Universal System Exclusive	25
8.1	Recognised messages	25
8.2	Messages not recognised	25
8.3	Practical consequences	25
8.4	The identifier is checked once	26
9	Settings that affect System Exclusive	27
9.1	The EXCLUSIVE filter	27
9.2	Which sockets carry what	28
9.3	Memory protect	28
10	Errors	29
10.1	The messages	29
10.2	ERROR 40 — wrong identification	29
10.3	ERROR 41 — reception	29
10.4	ERROR 42 — transmission	30
10.5	Interference from other traffic	30
11	SX-WSA1 and SX-WSA1R	31
12	Relationship to other Technics instruments	33
12.1	What is shared	33
12.2	What distinguishes the products	33
12.3	Dump regions differ	34
13	Worked examples	35
13.1	A parameter, written and read back	35
13.2	A frame from a real instrument	35
14	Scope of this edition	36

Chapter 1

Overview

This is unofficial documentation

Nothing here comes from Technics or Panasonic, and nothing here has been checked by them. This document was written by inspecting a disassembly of the SX-WSA1R's program ROMs, operating system version 2, and by confronting what that showed with a bulk dump captured from a real instrument.

Technics did publish a specification of its own, and where that specification covers the same ground this document follows it and uses its terms — section 1.3 says what it is and where it differs. Everything beyond it is inference from a disassembly, and inference can be wrong. Treat a statement here as a report of what one machine was observed to do, not as a guarantee about yours.

The Technics SX-WSA1R exchanges data over MIDI System Exclusive. This reference describes the messages it sends and accepts: their byte layout, the bulk-dump transfer protocol, the universal messages it honours, and what it reports when a transfer fails.

1.1 Applicability

This reference describes operating system version **2.0**. An earlier version 1 exists; its behaviour may differ and is not covered.

1.2 Ports

The instrument has two sets of MIDI sockets, MIDI 1 and MIDI 2. Ordinary performance data is handled on both. **Bulk dump transfers use MIDI 1 only** — a dump offered on MIDI 2 is not received, and the instrument sends dumps only from MIDI 1.

1.3 Relationship to the published documentation

Technics documented this instrument's System Exclusive implementation, in the *Reference Guide* that shipped alongside the two user manuals. Its pages 34–35 carry a MIDI implementation chart headed *Synthesizer* [SX-WSA1] *Synthesizer module* [SX-WSA1R], and pages 41–56 give the message format, the address map and the parameter tables. Where that guide and this reference describe the same thing, the guide's terms are used here and its statements are followed.

This reference exists because the published specification describes the *format* and not the *behaviour*. It does not say what raises each error number, that a dump can be requested only

while one screen is displayed, that the EXCLUSIVE filter governs a single message, that the rack refuses two functions silently, or that the instrument stops announcing its own General MIDI changes once a controller has sent one. Those are the subject of the chapters that follow, and they were established by decoding the program and by checking a dump from a real machine.

An earlier edition of this reference said no chart was published

It said the instrument shipped without a MIDI implementation chart. That was wrong: the chart and the full message format are in the *Reference Guide*, which is a separate volume from the user manual the claim had been checked against. The document now takes the published specification as its starting point.

One statement of the published documentation is contradicted here: the meaning of ERROR 42, set out in chapter 10.

1.4 Verification

The message layouts, the payload encoding and the checksum rule in this reference have been checked against a bulk dump captured from a real instrument: 2 883 messages. Despite its name the capture is a three-category session — SYSTEM, PART & MIDI, SOUND and COMBINATION — which matters, because the first of those is the category chapter 7 maps, so the parameter tables can be read against it directly. Of these, 2 876 carry a checksum, and all 2 876 verify under the rule given in chapter 5. Every transfer in the capture delivered exactly the byte count its header declared.

The parameter placement in chapter 7 has been checked against the same capture. The SYSTEM, PART & MIDI category was reassembled from it and each parameter’s own field read out of the result, for all 32 parts: 1 833 fields, of which 1 832 hold a value inside the range the instrument accepts for that parameter. Run against a plausible but wrong placement — one uniform stride for all 32 parts, instead of the two runs chapter 7 describes — the same check puts 69 fields out of range, so it is capable of rejecting a wrong map.

What the capture does not cover

Three of the four bulk-dump categories appear in it. SEQUENCER does not, and cannot: the capture came from a rack, which refuses that category (chapter 11). Everything this reference says about SEQUENCER — its three blocks, their addresses and lengths, and the variable length of the last one — rests on the program alone and has not been confronted with hardware.

The same applies to anything only a keyboard can do. A reader with an SX-WSA1 can close that gap with one transfer.

Two fields in that capture label themselves. Every part’s MIDI channel holds its own part number, so the 32 parts spell out an unbroken ramp from 0 to 31 — which the wrong placement interrupts at part 8, the first part past the gap. Every part’s pan holds 64, the value the instrument’s own scale displays as CTR. Those two names in chapter 7 therefore rest on a real dump as well as on the instrument’s screen.

1.5 Conventions

Byte values are hexadecimal. A field written *nn* takes one of the values listed for it. Ellipses inside a message stand for a body whose length depends on the message.

Every message begins with **F0** and ends with **F7**, as required by MIDI.

Real-time messages are safe inside a transfer

System Real Time bytes (**F8–FF**) may be interleaved anywhere inside a System Exclusive message without disturbing it. Conversely, while the instrument is receiving a System Exclusive message it ignores **F8** TIMING CLOCK, **FA** START, **FB** CONTINUE and **FC** STOP for transport purposes.

Chapter 2

At a glance

Everything in this chapter is stated again, with its conditions, in the chapters that follow. This page is for someone who already knows the protocol and wants the bytes.

2.1 Frame

F0 50 CMD 04 *md* 11 [ADR×3 SIZ×3 DT... *cn sm*] F7

04 is PC, the product; *md* is 00 for the SX-WSA1 and 01 for the SX-WSA1R, and both are accepted by both; 11 is VER, the exclusive specification's version. ADR and SIZ are 21-bit values in three seven-bit bytes, most significant first. There is no device or unit number anywhere.

2.2 Messages

F0 50 21 04 <i>md</i> 11 F7	HRQ	hand shake request
F0 50 22 04 <i>md</i> 11 F7	HRT	hand shake routine
F0 50 23 7E F7	ACK	acknowledge
F0 50 24 7E F7	NAK	not acknowledged
F0 50 25 <i>lo hi</i> F7	TMP	tempo, <i>lo</i> + 16 <i>hi</i> ; no checksum
F0 50 27 7E F7	EOK	end of category
F0 50 28 7E F7	END	end of dump
F0 50 29 7E F7	ERR	abort
F0 50 2A 7E F7	FUL	destination full
F0 50 2B ...	DRQ	request: ADR, SIZ, no DT
F0 50 2C ...	ITR	write one parameter
F0 50 2D ...	BTR	bulk data, first frame
F0 50 7E ...	CDD	bulk data, every frame after
F0 7E 7F 09 01 F7		General MIDI System On
F0 7E 7F 09 02 F7		General MIDI System Off

2.3 Rules

Payload. Each source byte goes out as two bytes, **high nibble first**: F1 becomes 0F 01. A payload is always an even number of bytes.

Checksum. $sm = (0 - S) \& 7F$, where S sums every byte from 50 through cn . Equivalently everything after F0, checksum included, sums to zero modulo 128. Verified on reception for 7E, 2B, 2C and 2D only.

Continue. cn is 01 while more frames follow, 00 on the last.

Size. At most 256 bytes per message including F0 and F7: 120 source bytes in a BTR frame, 125 in a CDD frame.

2.4 Bulk dump

ADR	area	menu name
40 00 00	PANEL	SYSTEM, PART & MIDI
20 00 00	SOUND MEMORY	SOUND
50 00 00	COMBINATION	COMBINATION
60 00 00	SEQUENCER	SEQUENCER (keyboard only)

Send DRQ with one of those to ask for a category. The instrument must be showing the SYSEX BULK DUMP screen to receive a dump, and bulk traffic works on the MIDI 1 sockets only.

2.5 Errors

ERROR 40 wrong product, model or version. ERROR 41 anything wrong with reception, including a missing acknowledgement. ERROR 42 an internal deadline, not a fault at the far end. ERROR 21 the destination is full.

Chapter 3

Message framing

3.1 Identifiers

The instrument accepts exactly two System Exclusive identifiers:

identifier	meaning
50	Matsushita (Technics)
7E	Universal Non-Real Time

A message carrying any other identifier is tracked to its F7 and discarded. No error is reported for it.

There is no device or unit number after the identifier

Many instruments place a device id immediately after the manufacturer identifier. The SX-WSA1R does not. The byte following 50 is already CMD, the command, described in chapter 4. A model byte does appear two positions later in most commands, but it names the model, not the device, and it is not where the convention would put a device id.

3.2 Termination

A message ends at F7. If any other status byte in the range 80–F6 arrives while a message is open, the message is abandoned and its contents discarded.

3.3 Length limit

A complete message may not exceed 256 bytes counting F0 and F7, that is 254 bytes between them. A longer message is abandoned and, inside a bulk-dump session, reported as a reception error. Dumps sent by the instrument use the full 256 bytes for all but the last message of each transfer.

Chapter 4

The Technics message set

4.1 Fields

Technics' published specification names the fields of a Technics System Exclusive message. This reference uses those names.

field	value	meaning
SOX	F0	exclusive status
IDC	50	Technics identifier
CMD		command, see below
PC	04	product category, WSA
	7E	DMY, the dummy used where no product is named
MD	00	model, SX-WSA1
	01	model, SX-WSA1R
VER	11	exclusive version 2.1
<i>then, for the messages that carry data:</i>		
ADR	3 bytes	a 21-bit address, seven bits each, most significant first
SIZ	3 bytes	length in source bytes, encoded the same way
DT		the data
CN	00/01	continue identifier
SM		checksum
EOX	F7	end of exclusive

VER is a version, not part of a model identifier

11 is the version of the *exclusive specification*, 2.1. It is not an operating system version — this instrument's operating system 2.0 sends VER = 11 — and it is not part of a model code. The model is MD, the byte before it.

4.2 Commands

After F0 50, CMD selects the message.

CMD		
21	HRQ	hand shake request
22	HRT	hand shake routine
23	ACK	acknowledge
24	NAK	negative acknowledge
25	TMP	tempo data
27	EOK	end of block
28	END	end
29	ERR	error
2A	FUL	memory full
2B	DRQ	data request
2C	ITR	individual data
2D	BTR	data block
7E	CDD	continuing data

The instrument accepts one further value that the published table does not list, 7F, and does not accept 26.

4.3 Which fields each message carries

ACK, NAK, EOK, END, ERR and FUL name no product: they carry PC = 7E DMY and stop there, with no MD and no VER. That is why they are five bytes long.

message		
F0 50 23 7E F7	ACK	
F0 50 24 7E F7	NAK	
F0 50 27 7E F7	EOK, end of block	
F0 50 28 7E F7	END, end of dump	
F0 50 29 7E F7	ERR	
F0 50 2A 7E F7	FUL, memory full	
F0 50 21 04 <i>md</i> 11 F7	HRQ, hand shake request	
F0 50 22 04 <i>md</i> 11 F7	HRT, hand shake routine	

DRQ, ITR and BTR carry PC, MD and VER, then ADR and SIZ, then CN and SM. ITR and BTR carry DT between them; DRQ, which asks rather than tells, carries none. CDD carries DT, CN and SM with no address at all.

MD is rewritten on transmission

The keyboard transmits MD = 00, the rack 01. The rack substitutes the byte as the message leaves, on every 21, 22, 2C and 2D message it sends, and recomputes the checksum afterwards, so the substitution is invisible to a receiver that validates it. A bulk dump captured from a real rack carries 04 01 11 throughout.

Reception is model-blind. Both values are accepted by both models, on every command that carries MD, and reach the same handler; a message is never refused, ignored or treated differently because of it. A receiver must therefore accept both, and a file dumped from one model loads into the other.

A message whose PC, MD or VER the instrument does not recognise is rejected with ERROR 40.

4.4 Tempo

TMP carries a tempo and nothing else:

F0 50 25 *lo hi* F7

Six bytes. Unlike every other Technics command it carries **no PC, no MD, no VER and no checksum**, and any bytes sent beyond the fifth are ignored. The tempo is nine bits sent low nibble first: $value = lo + 16 hi$. The accepted range is 40 to 300 beats per minute and a value outside it is treated as 120.

This message is the one thing the EXCLUSIVE filter governs; see chapter 9. It is also one of the two things the rack does not support; see chapter 11.

4.5 Parameter commands

DRQ requests and ITR writes an individual parameter. Both address the space chapter 7 sets out; chapter 13 works one through byte by byte.

Chapter 5

Bulk dump

Bulk dump transfers the instrument’s internal data as System Exclusive. The instrument is silent for the duration, and the transfer uses the MIDI 1 sockets only.

5.1 Categories and what each transmits

The SYSEX BULK DUMP screen offers five categories. Each sends one or more BTR messages at fixed addresses. The menu’s names and the published address map’s names for the same data differ; both are given.

category	area	ADR	SIZ	length	blocks
SYSTEM, PART & MIDI	PANEL	40 00 00	00 00 20	32	
		40 00 20	00 12 60	2 400	
SOUND	SOUND MEMORY	20 00 00	10 00 00	262 144	32
COMBINATION	COMBINATION	50 00 00	00 06 00	768	
		50 06 00	05 40 00	90 112	16
SEQUENCER	SEQUENCER	60 00 00	00 18 00	3 072	
		60 18 00	01 70 00	30 720	
		62 08 00	set at send time		
TOTAL KEYBOARD	the four categories in the order above				

Lengths are in source bytes; on the wire each occupies twice as many, because of the payload encoding described below.

The *blocks* column is internal structure, not extra messages: SOUND sends one header covering all 262 144 bytes, which is 32 repeats of an 8 192-byte layout, and COMBINATION’s second block one header covering 90 112 bytes as 16 repeats of 5 632. Those are the instrument’s 32 sounds and 16 combinations.

TOTAL KEYBOARD runs the four categories one after another in the order listed, each closed by its own end-of-category message, with a single end-of-dump message after the last.

The transmit order is not the menu order

The table above is in transmit order, which a capture from a real instrument confirms: SYSTEM, PART & MIDI goes first. The SYSEX BULK DUMP menu lists the same categories in a different order, with SOUND at the top. A receiver should take each transfer's ADR as the statement of what is arriving, and not infer the category from its position in the session.

The SEQUENCER category is the only one whose final message has no fixed length — it carries as much recorded material as the instrument holds.

The address is an exclusive address, not a memory address

ADR is a position in the published MIDI exclusive address map, which chapter 7 gives in full. It is not an address inside the instrument: SOUND and COMBINATION are held side by side in the machine, yet their exclusive addresses are 786 432 apart. For a bulk dump it is best treated as a fixed name. On reception the instrument compares ADR and SIZ against a fixed list and performs no arithmetic on them, so seven of the eight blocks are pinned on all six bytes; see below.

5.2 Data message — BTR

F0 50 2D 04 *nn* 11 *a2 a1 a0 l2 l1 l0 data... c sum* F7

nn MD, the model: 00 or 01.

a2 a1 a0 ADR, the destination, three bytes, most significant first, seven bits each: the value is $a_2 \times 128^2 + a_1 \times 128 + a_0$.

l2 l1 l0 SIZ, the length in *source* bytes, encoded the same way.

data DT, the payload, encoded as described below.

c CN, the continue identifier: 01 if further messages follow, 00 on the last.

sum SM, the checksum.

How much each frame carries

A complete message may not exceed 256 bytes including F0 and F7 (chapter 3), and the two frame shapes reach that limit differently, because a BTR header is eleven bytes where a CDD continuation header is two:

frame	total	payload bytes	source bytes
BTR, the first of a transfer	255	240	120
CDD, every one after it	256	250	125

Longer transfers are split across as many CDD frames as needed. Those figures are the observed maxima in a dump from a real instrument, not a stated limit; a sender may use less.

5.3 Payload encoding

Each source byte is sent as two message bytes, **high nibble first**:

$$b \longrightarrow (b \gg 4), \quad (b \& 0F)$$

So source byte **F1** appears on the wire as **0F 01**, and the payload occupies twice as many bytes as the length field states. A receiver rejoins them as $((\text{first} \& 0F) \ll 4) | (\text{second} \& 0F)$.

An odd number of payload bytes is invalid.

5.4 Checksum

Let S be the eight-bit sum of every byte from the identifier **50** through the continuation flag inclusive. Then

$$\text{sum} = (0 - S) \& 7F$$

The leading **F0**, the checksum byte itself and the trailing **F7** take no part in the sum. Equivalently, the sum of every byte after **F0** including the checksum is zero modulo 128.

The checksum is not verified for every family

On reception the checksum is checked only for families **7E**, **2B**, **2C** and **2D**. For the other accepted families a wrong checksum passes unremarked.

5.5 Requesting a dump — DRQ

A dump can be asked for over MIDI; the front-panel **SEND** button is not the only way to start one. The request names a category by the top byte of **ADR**:

F0 50 2B 04 nn 11 area 00 00 x x x F7

ADR	category requested
40	SYSTEM, PART & MIDI
20	SOUND
50	COMBINATION
60	SEQUENCER

The three bytes of **siz** are ignored; any values are accepted. The instrument then runs exactly the transfer the corresponding menu row would have run.

There is no request for **TOTAL KEYBOARD**: the four categories must be asked for one at a time.

5.6 Opening a transfer

A dump begins with a two-step exchange. Each step is attempted up to three times.

1. The sender transmits **F0 50 21 04 00 11 F7** and expects **F0 50 22 04 nn 11 F7**.
2. The sender transmits **F0 50 22 04 00 11 F7** and expects **F0 50 23 7E F7**.

If the exchange succeeds, every subsequent data message must be acknowledged. If all three attempts of the first step fail, **the dump proceeds anyway, without acknowledgements**, at a fixed pace of roughly 51 ms between messages.

5.7 Acknowledgement and retry

With acknowledgements in force, each data message must be answered with F0 50 23 7E F7. Any other reply causes the same message to be retransmitted, up to three attempts, after which the transfer is abandoned.

A reply is awaited for about 5.1 seconds; during the opening exchange the wait is about 2.05 seconds.

5.8 Continuation — CDD

Data messages after the first open with F0 50 7E, CDD, followed by DT, CN, SM and F7. A continuation carries no address.

5.9 Closing

At the end of each category the sender transmits F0 50 27 7E F7 and waits for an acknowledgement. At the end of the whole transfer it transmits F0 50 28 7E F7. If the transfer was abandoned after three failed retries it transmits F0 50 29 7E F7.

5.10 Receiving

When the SX-WSA1R is the receiver it answers each data message with F0 50 23 7E F7 to accept, F0 50 24 7E F7 to reject, and F0 50 2A 7E F7 when the incoming data is larger than the destination.

While a bulk-dump session is open the instrument accepts only the 50 identifier; universal messages are not honoured until the session ends.

The instrument must be on the SYSEX BULK DUMP screen to receive

A dump cannot be pushed to an idle instrument. Reception happens only while that screen is displayed; a dump sent at any other time is parsed, discarded, and not reported.

A dump offered on the MIDI 2 input is never received either, at any time. This is what lies behind the published instruction to use MIDI 1 for bulk transfers.

Seven of the eight patterns are pinned on all six bytes, address and length together. Sending a correct address with a shortened length is not a short transfer — it is an unrecognised message, refused with ERROR 41. Only SEQUENCER block 3 has its length read as a number, because that block's size varies. An arbitrary destination cannot be written.

The one length the message itself controls is SEQUENCER part 3, which is capped at 330 752 bytes; beyond that, and whenever the destination fills mid-message, the instrument answers F0 50 2A 7E F7 and reports ERROR 21.

Memory-full is treated as end-of-dump

When the SX-WSA1R is *sending* and the receiver answers F0 50 2A 7E F7 to report that its memory is full, the instrument treats that exactly as it treats F0 50 28 7E F7, end of dump. It stops, and it reports success. A sender that needs to know whether the data arrived cannot rely on the instrument to notice.

Chapter 6

What a block contains

This reference describes the protocol that carries a dump. The meaning of the data inside a block is outside its scope, with one exception worth stating: every block opens with a fixed signature, and two of them carry readable names. Together these let a program check a stored dump before sending it back, and list what is in it, without understanding the rest.

6.1 Signatures

The bytes below are the first bytes of the reassembled block — after the payload has been rejoined from its nibbles, not as they appear on the wire. They were read out of a dump from a real instrument, and the two rows of each entry are the same sixteen bytes shown twice, as hexadecimal and as text.

block	ADR	first sixteen bytes	as text
SYSTEM, PART & MIDI 1	40 00 00	5A 5A 01 00 57 41 30 00 00 00 A0 00 00 00 00 00	ZZ..WA0.
SYSTEM, PART & MIDI 2	40 00 20	78 10 20 20 20 20 20 20 20 20 20 20 20 20 20 20	x.
SOUND	20 00 00	57 53 41 20 53 4F 55 4E 44 20 52 41 4D 20 53 30	WSA SOUN D RAM SO
COMBINATION 1	50 00 00	5A 5A 5A 5A 00 00 57 53 41 31 20 20 01 00 00 02	ZZZZ..WS A1
COMBINATION 2	50 06 00	78 10 20 53 70 61 63 65 20 49 6E 66 69 6E 69 74	x. Space Infini

A file whose first bytes do not match the block it claims to be is not a dump of that category.

6.2 Names

The SOUND block continues, immediately after its sixteen-byte signature, with sixteen-byte names in the instrument’s own character set, padded with spaces. The second COMBINATION block and the second SYSTEM, PART & MIDI block use a different framing: a two-byte prefix, then a sixteen-byte name, then two further bytes.

A librarian can therefore show the user what a stored file holds — “SWEEP PAD”, “Space Infinity” — without decoding anything else.

Reassembly is self-checking

A block reassembled correctly comes out at exactly the length its own header declared. A wrong nibble order, or a payload taken with its trailing flag and checksum still attached, does not. Checking the length is a cheap way to confirm a receiver is unpacking correctly before trusting anything it produces.

Chapter 7

Parameter messages

Two commands address individual parameters rather than transferring blocks. ITR (2C) writes one; DRQ (2B) requests one.

F0 50 2B 04 *md* 11 *a2 a1 a0 s2 s1 s0 cn sm* F7

7.1 The address

ADR is a single 21-bit address, sent as three seven-bit bytes, most significant first. It is the same address space the bulk dump uses — a dump and a parameter write are two ways of reaching one map. The published map divides it as follows.

ADR	address	area	subarea	
00 00 00	000000	SYSTEM		real time
00 08 00	000400	SYSTEM		non-real time
00 10 00	000800	PART	COMMON	real time
00 18 00	000C00	PART	COMMON	non-real time
00 20 00	001000	PART	INDIVIDUAL	part 1, real time
00 20 40	001040	PART	INDIVIDUAL	part 1, non-real time
00 3F 40	001FC0	PART	INDIVIDUAL	part 32, non-real time
00 60 00	003000	PART	SPECIAL	real time
00 68 00	003400	PART	SPECIAL	non-real time
10 00 00	040000	NORMAL SOUND	INDIVIDUAL	real time
18 00 00	060000	DRUM SOUND	INDIVIDUAL	real time
20 00 00	080000	SOUND MEMORY	TOTAL	
40 00 00	100000	PANEL	HEADER	
40 00 20	100020	PANEL	PANEL DATA	
50 00 00	140000	COMBINATION	HEADER	
60 00 00	180000	SEQUENCER	LOCATION	

The last five are the bulk-dump categories of chapter 5. The ones above them are what an individual parameter message reaches.

7.2 Parts

A part occupies 80 bytes of the map, split in half: the first 40 are its real-time parameters and the second 40 its non-real-time ones. So for part *n*, counting from 1, the middle byte of

ADR is $20 + n - 1$, and the last byte is the parameter, below 40 for a real-time parameter and 40 or above for a non-real-time one.

All 32 parts take the same 57 parameters.

7.3 What the instrument actually accepts

Not every address in the map is a parameter. In the 00 area the instrument accepts 108 distinct addresses, which the tables below enumerate in full.

ADR middle	area	subarea	parameters
00, 01	SYSTEM		22
08	SYSTEM	non-real time	4
10, 11	PART	COMMON	24
20–3F	PART	INDIVIDUAL	57 each
60	PART	SPECIAL	1

Under 11, values of the last byte from 03 upward are admitted in three runs, one per effect processor:

run	processor	
20–36	EFFECT1	type, twenty values, send, dynamic control
40–56	EFFECT2	type, twenty values, send, dynamic control
60–75	REVERB	type, twenty values, dynamic control

Within a run the first address is the effect type and the next twenty are its parameters, whose meaning depends on the type selected. Any value of the last byte outside the three runs is accepted by the parser and then discarded, producing no error and no effect.

7.4 Parameters and the bulk dump are the same bytes

The PART and SYSTEM parameters and the PANEL bulk dump — the category the menu calls SYSTEM, PART & MIDI — are two views of one object. That dump carries 100000 to 100980, and every parameter of the 00 area is one byte, or a field of one byte, inside it.

This matters in both directions. An editor that writes parameters and a librarian that restores dumps are competing for the same storage, so a dump restored after a parameter write discards that write. It also gives a way to confirm any row of the tables below without opening the instrument: dump the category, change the one parameter, dump again, and the two dumps differ at the stated address and nowhere else.

Where a part’s record sits

Each part occupies a 40-byte record in that dump, and the offsets in table 7.1 are relative to the start of it. The records are in two runs:

part	record begins at
1 to 8	$1000A2 + 40 \times (\text{part} - 1)$
9 to 32	$1002E2 + 40 \times (\text{part} - 9)$

The part records are not one array

The obvious formula, $1000A2 + 40 \times (\text{part} - 1)$, is correct for the first eight parts and wrong for the other twenty-four. A 40-byte gap after part 8 holds common-block storage, and the parts resume above it. Code that assumes a single stride will read and write the wrong part for parts 9 upward, and will corrupt two common blocks on the way past.

7.5 What a parameter accepts

Each parameter has a fixed data length and a range of accepted values. A value outside the range is rejected without an error message. The *bits* column gives which bits of the dump byte belong to the parameter; where that is not all of them, other parameters share the byte, and a dump editor changing one must preserve the rest.

Three parameters accept an enumerated set rather than a range; sending any other value has no effect.

Two parameters where the published range and the range check disagree

The *values* column below is the range the instrument's own check enforces. For all but two parameters that is also the published range. The exceptions:

00 00 08 **MASTER TUNING**. Published as C0–00–3F, a signed value with 00 at concert pitch, spanning 427.3 to 453.0 Hz. The range check admits 48 to 192. A dump from a real machine holds 00 there — the published centre, and not a member of the checked range.

00 *nn* 05 **EFFECT2 ON/OFF**. Published as 00–01. The range check admits 0 to 127, and the instrument passes the value through a substitution before storing it, so reading it back does not return what was written.

Send what the published specification gives.

The accepted values are what a write is checked against

They are what the instrument takes from an ITR, not a guarantee about what a stored byte contains. A librarian should not validate a dump by checking it against these ranges.

Table 7.1: The 57 parameters of a part block. *p* is the third address byte; the offset is the byte's position inside that part's record in a SYSTEM, PART & MIDI bulk dump.

<i>p</i>	data	offset	bits	values	name
00	3	00	all	0–127	PROGRAM CHANGE & BANK
03	1	03	7F	0–127	VOLUME
04	1	05	7F	0–127	EFFECT1 SEND
05	1	06	7F	0–127	EFFECT2 ON/OFF
06	1	07	7F	0–127	REVERB SEND
07	1	08	7F	0–127	PANPOT
08	1	09	7F	28–100	KEY SHIFT

<i>p</i>	data	offset	bits	values	name
09	1	0A	all	0–255	FINE TUNE
0A	1	0B	7F	0–12	PITCH BEND RANGE
10	1	26	0F	0–1	ASSIGN MODE
14	1	0C	08	0–1	KEY SCALING
20	1	25	all	0–48	VELOCITY OFFSET
28	1	27	7F	0–127	KEY LAYER LOW
29	1	28	7F	0–127	KEY LAYER HIGH
2A	1	29	7F	0–127	VELOCITY LAYER LOW
2B	1	2A	7F	0–127	VELOCITY LAYER HIGH
30	1	23	all	0–1	MAIN OUT
31	1	24	all	enumerated, 4 values	SUB OUT
40	1	0D	20	0–1	LOCAL CONTROL
41	1	0D	1F	0–31	BASIC CHANNEL
42	1	0D	40	0–1	MIDI OUT SETTING
43	1	0D	80	0–1	MIDI IN SETTING
48	1	2F	10	0–1	MIDI INPUT FILTER: PROGRAM CHANGE
49	1	30	80	0–1	MIDI INPUT FILTER: BANK SELECT
4A	1	30	04	0–1	MIDI INPUT FILTER: VOLUME
50	1	33	10	0–1	MIDI OUTPUT FILTER: PROGRAM CHANGE
51	1	34	80	0–1	MIDI OUTPUT FILTER: BANK SELECT
52	1	34	04	0–1	MIDI OUTPUT FILTER: VOLUME
53	1	34	10	0–1	MIDI OUTPUT FILTER: PANPOT
54	1	34	20	0–1	MIDI OUTPUT FILTER: EFFECT DEPTH
55	1	33	40	0–1	MIDI OUTPUT FILTER: PITCH BEND
56	1	34	02	0–1	MIDI OUTPUT FILTER: MODULATION1
57	1	36	10	0–1	MIDI OUTPUT FILTER: MODULATION2
58	1	36	01	0–1	MIDI OUTPUT FILTER: REAL-TIME CREATOR-X
59	1	36	02	0–1	MIDI OUTPUT FILTER: REAL-TIME CREATOR-Y
5A	1	36	04	0–1	MIDI OUTPUT FILTER: REAL-TIME CONTROLLER-X
5B	1	36	08	0–1	MIDI OUTPUT FILTER: REAL-TIME CONTROLLER-Y
5C	1	34	01	0–1	MIDI OUTPUT FILTER: HOLD1
5D	1	36	20	0–1	MIDI OUTPUT FILTER: CONTROL PEDAL
5E	1	33	20	0–1	MIDI OUTPUT FILTER: AFTER TOUCH
60	1	0E	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: PROGRAM CHANGE
61	2	—	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: BANK SELECT
63	1	11	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: VOLUME
64	1	12	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: PANPOT
65	1	14	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: REVERB DEPTH
66	1	13	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: CHORUS DEPTH
68	1	37	all	28–100	MIDI OUT KEY TRANSPOSE
70	1	2B	40	0–1	CONTROLLER INTERNAL FILTER: PITCH BEND

<i>p</i>	data	offset	bits	values	name
71	1	2C	02	0–1	CONTROLLER INTERNAL FILTER: MODULATION1
72	1	2E	10	0–1	CONTROLLER INTERNAL FILTER: MODULATION2
73	1	2E	01	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CREATOR-X
74	1	2E	02	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CREATOR-Y
75	1	2E	04	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CONTROLLER-X
76	1	2E	08	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CONTROLLER-Y
77	1	2C	01	0–1	CONTROLLER INTERNAL FILTER: HOLD1
78	1	2E	20	0–1	CONTROLLER INTERNAL FILTER: CONTROL PEDAL
79	1	2B	20	0–1	CONTROLLER INTERNAL FILTER: AFTER TOUCH

Table 7.2: The parameters of the common blocks. The address is the one a SYSTEM, PART & MIDI bulk dump carries for the same byte.

<i>s</i>	<i>p</i>	data	address	bits	values	name
00	00	1	outside	01	0–1	SOUND MEMORY PROTECT
00	01	1	outside	02	0–1	COMBINATION MEMORY PROTECT
00	08	1	10094A	all	48–192	MASTER TUNING
00	10	1	10095A	0F	0–9	VELOCITY CURVE
00	11	1	10095C	7F	0–127	VELOCITY OFFSET
00	12	1	10095F	all	0–10	AFTER TOUCH CURVE
00	13	1	100960	7F	0–127	AFTER TOUCH THRESHOLD
00	20	1	10094E	all	enumerated, 4 values	DRUMS MAP SELECT
00	30	1	1008E4	0F	0–2	DATA LOAD FILTER: EFFECT & OUTPUT
00	31	1	1008E4	F0	0–2	DATA LOAD FILTER: REAL-TIME CREATOR 1-6
00	32	1	100907	04	0–1	DATA LOAD FILTER: COMBINATION OCTAVE
00	33	1	100907	02	0–1	DATA LOAD FILTER: MIDI SETTING
00	34	1	100907	01	0–1	DATA LOAD FILTER: KEY & VELOCITY LAYER
00	35	1	100907	08	0–1	DATA LOAD FILTER: MAIN OUT EQUALIZER
00	36	1	100907	10	0–1	DATA LOAD FILTER: KEY SCALING
00	40	2	—	7F	0–127	COMBINATION NUMBER and BANK
01	00	1	100935	0F	0–2	MIDI INPUT MODE
01	01	1	100935	F0	0–1	MIDI OUTPUT MODE
01	02	1	100936	1F	0–15	SINGLE CHANNEL
01	03	1	100936	20	0–1	LOCAL TOTAL

<i>s</i>	<i>p</i>	data	address	bits	values	name
01	04	1	100932	03	0–1	PROGRAM CHANGE MODE
01	05	1	100932	08	0–1	SINGLE CHANNEL PROGRAM CHANGE
08	00	1	—	—	0–6	INITIAL
08	08	1	—	F0	0–1	PLAY MODE REQUEST
08	10	1	—	—	0–127	COMBINATION WRITE REQUEST
08	11	3	—	—	0–1	SOUND WRITE REQUEST
10	00	1	1002B2	7F	28–100	KEY TRANSPOSE
10	10	1	1002A3	80	0–1	KEY SCALING MODE
10	11	1	1002A2	all	enumerated, 15 values	TOTAL KEY SCALING
10	12	1	1002A3	0F	0–11	KEY SCALING SHIFT
10	13	1	1002A4	all	0–255	Ckey DETUNE SETTING
10	14	1	1002A5	all	0–255	Dbkey DETUNE SETTING
10	15	1	1002A6	all	0–255	Dkey DETUNE SETTING
10	16	1	1002A7	all	0–255	Ebkey DETUNE SETTING
10	17	1	1002A8	all	0–255	Ekey DETUNE SETTING
10	18	1	1002A9	all	0–255	Fkey DETUNE SETTING
10	19	1	1002AA	all	0–255	Gbkey DETUNE SETTING
10	1A	1	1002AB	all	0–255	Gkey DETUNE SETTING
10	1B	1	1002AC	all	0–255	Abkey DETUNE SETTING
10	1C	1	1002AD	all	0–255	Akey DETUNE SETTING
10	1D	1	1002AE	all	0–255	Bbkey DETUNE SETTING
10	1E	1	1002AF	all	0–255	Bkey DETUNE SETTING
10	20	1	—	1F	0–17	MAIN OUT EQUALIZER LOW-FREQ
10	21	1	1002B3	3F	0–48	MAIN OUT EQUALIZER LOW-GAIN
10	22	1	—	1F	17–26	MAIN OUT EQUALIZER HIGH-FREQ
10	23	1	1002B5	3F	0–48	MAIN OUT EQUALIZER HIGH-GAIN
11	00	1	100034	01	0–1	EFFECT COMMON ALGORITHM
11	01	1	1002B7	0F	1–4	EFFECT1 OUTPUT SELECT
11	02	1	1002B7	F0	1–4	EFFECT2 OUTPUT SELECT
11	20	1	—	—	0–0	EFFECT1 TYPE
60	00	1	100905	7F	0–127	METRONOME VOLUME

parameter	accepted values	count
00 00 20	00 40 41 42	4
00 10 11	00 40 41 42 03 04 06 10 11 12 13 14 15 16 80	15
00 20 31	00 02 03 04	4

7.6 Names

Every name in the tables is the one Technics’ own parameter tables print for that address. None is inferred. Two of them are corroborated a second way, from a dump taken off a real machine: every part’s PANPOT reads 64, the value the instrument displays as CTR, and the 32 parts’ BASIC CHANNEL bytes read an unbroken ramp from 0 to 31.

The 08 block is marked receivable but not requestable: the instrument accepts a write to

it and will not report its contents. Its four addresses are commands rather than settings — they initialise a section, switch play mode, or ask for a store.

7.7 Whole-category request

DRQ also requests an entire bulk-dump category, as described in chapter 5. There ADR names one of the lower five rows of the map above.

7.8 The sound areas

ADR values beginning 10 and 18 reach NORMAL SOUND and DRUM SOUND, the individual sound being edited rather than any stored one. This edition does not enumerate their parameters.

Chapter 8

Universal System Exclusive

8.1 Recognised messages

The instrument honours two universal messages, and transmits both.

message	meaning
F0 7E 7F 09 01 F7	General MIDI System On
F0 7E 7F 09 02 F7	General MIDI System Off

Turning General MIDI on restricts the sounds and operations available and may rearrange the percussion map. General MIDI System Off has no effect if General MIDI is already off.

A received message takes effect at once. The same change made at the front panel does not: the panel asks for a confirmation first, and the user can decline it. A controller sending General MIDI System On is therefore doing something the user cannot be offered the chance to refuse, and the mode change it causes is one the instrument warns will make material recorded in the other mode play back wrongly.

Both messages are also written into Standard MIDI Files produced by the instrument, as a five-byte System Exclusive event at delta time zero.

8.2 Messages not recognised

A controller expecting any of the following will receive no response:

message	
F0 7F 7F 04 01 <i>ll mm</i> F7	Master Volume
F0 7E <i>dd</i> 06 01 F7	Identity Request
F0 7E 7F 09 03 F7	General MIDI 2 System On

The instrument does not implement Identity Reply, so it cannot be auto-detected by a controller that relies on it. The entire Universal Real Time identifier 7F is unrecognised.

8.3 Practical consequences

A General MIDI message sent during a bulk dump breaks the dump. While a transfer is open the instrument accepts only messages beginning F0 50. One beginning F0 7E is treated as a reception failure: the transfer is abandoned and ERROR 41 is reported. General MIDI mode is left unchanged.

The instrument announces its own changes. Entering or leaving General MIDI mode makes it transmit the matching message, for a change made at the front panel as well as one made by loading a song. Neither direction is affected by the EXCLUSIVE filter.

The announcement stops for good once you send one

The interlock that stops the instrument echoing a General MIDI message back is armed by the first such message it receives after power-on, and is never disarmed. Once a controller has sent either message, the instrument stops announcing its own General MIDI changes for the rest of the session. A controller that tracks the instrument's mode by listening for these messages will silently stop being told.

General MIDI state travels in a dump. It is carried in the second SYSTEM, PART & MIDI block, so restoring that category restores the General MIDI setting with it. A dump taken in General MIDI mode is not the same file as one taken out of it.

8.4 The identifier is checked once

Once 50 or 7E has been accepted the instrument does not look at it again; the third byte alone selects the message. F0 50 7F 09 01 F7 is therefore acted on as General MIDI System On despite carrying the Matsushita identifier, and a Technics message sent with 7E in the identifier position works equally well. Send the conventional identifier, but expect a receiver to be lenient.

Nothing else about General MIDI mode changes the System Exclusive implementation: every family is accepted exactly as before, with the same encoding, checksum, handshake and error reporting.

Chapter 9

Settings that affect System Exclusive

9.1 The EXCLUSIVE filter

The MIDI menu's INPUT & OUTPUT FILTER page has eight rows, each a plain ON/OFF switch for one kind of MIDI message: PROGRAM CHANGE, BANK SELECT, PITCH BEND, CONTROL CHANGE, RESET ALL CNT., CHANNEL PRESSURE, SONG SELECT and, last, EXCLUSIVE. There is one such switch and it governs both directions: the same setting is consulted when the instrument sends and when it receives. There is no separate input filter, no per-socket filter and no third state, and the page is the same on both models.

EXCLUSIVE governs the tempo message and nothing else

Despite its name, this switch does not turn System Exclusive on and off. Its entire effect on the messages in this reference is the family 25 tempo message: with EXCLUSIVE set to OFF the instrument neither sends one nor acts on one received, silently in both directions.

A bulk dump transfers normally with EXCLUSIVE set to OFF. Software that cannot complete a transfer should not look here for the cause.

This is worth knowing before telling a user to change the setting: the illustration of that page in Technics' own guide shows EXCLUSIVE switched OFF, so an instrument presenting that page as pictured is already able to send and receive every message in this reference.

Everything below is unaffected by the setting:

function	affected by EXCLUSIVE?
bulk dump, sent or received	no
dump requests	no
the opening enquiry and its reply	no
acknowledgements and the session control messages	no
parameter messages, and the replies and refusals to them	no
General MIDI System On / Off, sent or received	no
error reporting	no
tempo message (family 25)	yes

9.2 Which sockets carry what

The instrument has two sets of MIDI sockets. Only bulk-dump traffic is restricted to the first.

message	accepted on
bulk-dump data, continuation frames, handshake, acknowledgements	MIDI 1 only
dump requests	MIDI 1 only
parameter messages (families 2B, 2C)	MIDI 1 and MIDI 2
tempo message (family 25)	MIDI 1 and MIDI 2
General MIDI System On / Off	MIDI 1 and MIDI 2

MIDI channels are written with the socket set first: 1-01 to 1-16 and 2-01 to 2-16. No channel or device-number setting affects System Exclusive addressing — these messages carry no channel.

9.3 Memory protect

Two write-protect switches are consulted before an incoming message is acted on. A message they block never reaches the transfer machinery, and the instrument shows its own write-protection prompt rather than a System Exclusive error. A transfer that stops with that prompt on screen has not failed as a transfer.

Chapter 10

Errors

Transfer failures are reported on the SYSEX BULK DUMP screen. A System Exclusive message arriving while any other screen is displayed produces no visible error.

10.1 The messages

screen	meaning
COMPLETED!	the transfer finished
ERROR 40!	the message is addressed to a different product
ERROR 41!	reception failed
ERROR 42!	transmission failed
ERROR 21!	the destination is full

Not every failure is a 40, 41 or 42

When incoming data is larger than the destination area, the instrument reports ERROR 21! MEMORY FULL. Software that recognises only the three documented System Exclusive errors will mis-read a truncated restore as something else.

10.2 ERROR 40 — wrong identification

Raised when CMD is accepted but the bytes that follow it do not match the product. For a message addressed to the SX-WSA1R that means PC was not 04, or MD was neither 00 nor 01, or VER was not 11.

10.3 ERROR 41 — reception

The general reception failure. It covers:

- a message inside a session that does not begin F0 50;
- a status byte other than F7 arriving inside an open message;
- a framing, parity or overrun error on the MIDI input, or loss of active sensing;
- no reply within the reply window, including the acknowledgement that each data message must receive;

- a message longer than 254 bytes;
- an odd number of payload bytes;
- a continuation flag that is neither 00 nor 01;
- a checksum mismatch.

10.4 ERROR 42 — transmission

Raised when an outgoing transfer stalls inside the instrument: the send path waits for one block to be handed over internally, and reports ERROR 42 when that wait times out after about a second. In version 2.0 this is its only cause.

ERROR 42 is not a report about the receiving device

The published manual describes ERROR 42 as an error during transmission, with the data not having been received correctly. That is not what the instrument tests. The condition it reports is internal to the sending machine and can arise with nothing connected to the MIDI output at all.

A genuine failure at the far end looks different. No acknowledgement within the reply window, a framing, parity or overrun error on the input, and a transfer abandoned after its three retries all raise ERROR 41. Software diagnosing a failed transfer should read ERROR 41, not ERROR 42, as the sign of a problem on the cable or at the receiver.

10.5 Interference from other traffic

While a bulk-dump session is open, unrelated System Exclusive traffic arriving on the MIDI input can set the error state, and will then be reported when the session ends. Keep other sources off the MIDI 1 input for the duration of a transfer.

Chapter 11

SX-WSA1 and SX-WSA1R

The SX-WSA1 keyboard and the SX-WSA1R rack module run the same program and speak the same System Exclusive protocol. Two features are nonetheless available on the keyboard and not on the rack, in both directions, and the two models put different bytes in the model triple. Nothing else in the System Exclusive implementation depends on which model it is: the framing, the checksum, the handshake, the error reporting, the parameter address space and every other bulk-dump category are identical.

	SX-WSA1 (keyboard)	SX-WSA1R (rack)
message framing, checksum, handshake	yes	yes
SOUND bulk dump	yes	yes
COMBINATION bulk dump	yes	yes
SYSTEM, PART & MIDI bulk dump	yes	yes
dump requests	yes	yes
General MIDI On / Off	yes	yes
MD transmitted	00	01
MD accepted	both	both
SEQUENCER bulk dump	yes	no
tempo message (family 25)	yes	no

On the rack, a SEQUENCER dump sends nothing and says nothing

The rack's SYSEX BULK DUMP menu has no SEQUENCER row: it is not drawn, and the soft key that would select it does nothing. The category can still be reached over MIDI, by a dump request addressed to 60 00 00, and as part of TOTAL KEYBOARD. In both cases nothing is transmitted — not even the end-of-category message that closes every other transfer. A librarian waiting for data will simply time out.

On the rack the transfer-progress screen likewise lists only SYS,PART&MIDI, SOUND and COMBINATION.

Technics' own guide points the same way: the photograph it prints of the SYSEX BULK DUMP menu, the one showing all five rows, is captioned (WSA1) — the marker the guide uses throughout for what belongs to the keyboard.

The tempo message is refused in both directions on the rack: it will neither act on one received nor send one.

Telling the two apart over MIDI

MD in anything the instrument transmits is 00 on the keyboard and 01 on the rack — the values Technics’ published field table gives for the two models. An identity enquiry (F0 50 21 04 00 11 F7 or F0 50 22 04 00 11 F7) is answered with the model’s own value, whichever value the enquiry itself carried. A second, behavioural test is to request a SEQUENCER dump and see whether anything arrives.

Which machine is which

The two behaviours are selected by a hardware input read once at power-on; the program names neither model. The identification does not depend on that, because Technics published the answer: the field table gives MD = 00 for the SX-WSA1 and 01 for the SX-WSA1R, and the machine that reads the second value is the one that transmits 01.

The same guide prints SEQUENCER : WSA1 ONLY beside the SEQUENCER row of its address map. Its user manual marks the whole sequencer chapter SEQUENCER (WSA1) and carries the same (WSA1) marker on fifty-nine further passages. A model with no sequencer has nothing to put in a SEQUENCER dump and no use for a tempo message, which is exactly the pair of System Exclusive functions that machine refuses.

Chapter 12

Relationship to other Technics instruments

The message engine described in this reference is not unique to the SX-WSA1R. At least two other Technics instruments implement the same protocol: the SX-KN5000 and the SX-KN1500.

12.1 What is shared

All three use the 50 identifier, the same message families, the same short control messages, the same payload encoding and the same checksum rule. The fixed messages are byte-for-byte identical:

```
F0 50 23 7E F7  F0 50 24 7E F7  F0 50 27 7E F7  F0 50 28 7E F7  F0 50 29 7E
                  F7  F0 50 2A 7E F7
```

The error mapping is also nearly common: of the thirty-four status values, thirty-two produce the same report on the SX-WSA1R and the SX-KN5000.

12.2 What distinguishes the products

PC, MD and VER together are the discriminator. A message whose three bytes do not match the receiving instrument is refused with ERROR 40.

instrument	PC MD VER
SX-WSA1 / SX-WSA1R	04 00 11, 04 01 11
SX-KN5000	01 28 12
SX-KN1500	01 24 11, 01 25 11, 01 26 11

The split is legible once the fields are named: PC = 04 is the WSA product category and 01 the keyboards, MD separates models within a category, and VER is the exclusive specification's own version — 2.1 for the WSA1 and the KN1500, 2.2 for the KN5000.

A librarian that supports more than one of these instruments needs only to vary those bytes and the dump region addresses; the framing, handshake, encoding and checksum are common code.

12.3 Dump regions differ

The dump region addresses are per-product. The SX-KN5000 and SX-KN1500 share eleven of their thirteen region and length pairs with each other, but the SX-WSA1R shares none of its eight with either — its categories are its own.

The SX-WSA1R does not accept the GS-style messages its siblings do

The SX-KN5000 and SX-KN1500 additionally accept a small set of messages in the 42 12 parameter form, including the sequence 42 12 40 00 7F 00 41 used elsewhere in the industry as a reset. **The SX-WSA1R accepts none of them.** Software that resets a KN-series instrument that way and assumes the SX-WSA1R will follow will find it does not respond.

Chapter 13

Worked examples

Three things in this format are easy to get subtly wrong: the seven-bit address encoding, the high-nibble-first payload, and which bytes the checksum covers. The examples below are complete messages, and every byte in them is computed rather than transcribed.

13.1 A parameter, written and read back

The example is part 0's parameter 03, a one-byte value, set to 100.

	message
write	F0 50 2C 04 00 11 00 20 03 00 00 01 06 04 00 41 F7
request	F0 50 2B 04 00 11 00 20 03 00 00 01 00 4C F7
reply	F0 50 2C 04 00 11 00 20 03 00 00 01 06 04 00 41 F7
part 7	F0 50 2C 04 00 11 00 27 03 00 00 01 06 04 00 3A F7
three-byte	F0 50 2C 04 00 11 00 20 00 00 00 03 01 02 00 00 04 00 00 45 F7

The value 100 is 64, which DT carries as the two bytes 06 04. The three bytes before it are SIZ, and the 00 after it is CN. SM closes the message.

13.2 A frame from a real instrument

The bytes below open the first data message of a dump recorded from an SX-WSA1R, a transfer of three categories in one session:

F0 50 2D 04 01 11 40 00 00 00 00 20 ...

Reading it by the rules of chapter 5: CMD is 2D BTR; 04 01 11 are PC, MD and VER, the middle one being the rack's; then ADR 40 00 00 and SIZ 00 00 20, which together name block 1 of SYSTEM, PART & MIDI. The length decodes to 32 source bytes and the frame carries 32 of them.

The payload decodes to 5A 5A 01 00 57 41 30 00..., the signature section 6.1 gives for that block.

That block is the one chapter 7 maps: a dump of this category can be read against table 7.1 directly, which is the cheapest way to confirm a parameter address without an instrument to hand.

Chapter 14

Scope of this edition

This reference covers message framing, the bulk-dump transfer protocol, the parameter address space, the universal messages, error reporting and the differences between the two models. The following are not covered.

What the parameters do. Chapter 7 enumerates all 108 addresses of the 00 area with their name, length, bit mask, accepted values and position in a dump. It does not explain what each setting does to the sound; the user manuals do that.

The effect parameters. The twenty values under each of EFFECT1, EFFECT2 and REVERB mean different things for each effect type. The published *Reference Guide* gives them, on its DSP EFFECT pages, and this edition does not reproduce them.

The sound areas. NORMAL SOUND and DRUM SOUND are reachable but not enumerated.

Block contents. Chapter 6 gives each block's signature and where its names are, but not the meaning of the rest of the bytes — the structure of a sound, a combination or a sequence.

The SEQUENCER category, against hardware. Its protocol is described, but the capture used to check everything else came from a rack, which cannot send it. See chapter 1.

Operating system version 1. Everything here describes version 2.0.